Clase: primer proyecto por CLI

Duración: 1.5 h · **Carril de código** · **Semana 1 del plan**

Hoy creas tu primer proyecto de .NET 10 desde la terminal, entiendes qué archivo hace qué y compruebas con tus propios ojos lo que viste el lunes: que cada tipo es una cantidad fija de bytes y que un `int` se desborda.

Modo tutor: esta clase te da los comandos para preparar el proyecto y el enunciado de cada ejercicio, con pistas. El código de los ejercicios lo escribes tú. Si te atoras, pregúntame el *concepto*, no la solución.

Plan de la sesión

Minuto	Qué haces	Técnica
0–5	Repaso de las tarjetas de 0.1 Datos sin mirar	Repetición espaciada
5–25	Lees 10 Teoria de CSharp, secciones 1 a 3, contestando sus preguntas de repaso. **Si ya las leíste, usa este bloque para el laboratorio 0**	Lectura activa
—	Laboratorio 0: crear el repositorio `plan-dotnet` en GitHub y clonarlo (solo la primera vez)	Práctica guiada
25–40	Laboratorio 1: preparar `plan-dotnet`	Práctica guiada
40–50	Laboratorio 2: explorar el proyecto	Interrogación elaborativa
50–80	Laboratorio 3: ejercicios de tipos y desbordamiento	Predecir antes de ejecutar
80–90	Commit, explicación de 2 minutos, glosario	Técnica Feynman

---

Repaso (5 min)

Sin mirar, contesta en voz alta:

1. ¿Por qué un byte va de 0 a 255?
2. ¿Cuánto es `0xFF` en decimal?
3. ¿Cuántos bytes tiene un `int`?

Si fallaste alguna, repasa esa parte de 11 Clase 0.1 Datos mañana antes de empezar.

Laboratorio 0. Crear y clonar el repositorio `plan-dotnet` (15 min, solo la primera vez)

Si ya tienes `plan-dotnet` clonado en tu máquina, salta al laboratorio 1.

Primero, cuatro palabras

> **Definición para entrevista: ¿qué es un repositorio de Git?** Es una carpeta cuyo historial de cambios registra **Git**, el programa de control de versiones: guarda cada versión de tus archivos como un **commit** (una "foto" del proyecto con un mensaje que explica qué cambió), para que puedas ver quién cambió qué y cuándo, y regresar a cualquier versión anterior.

Palabra	Qué es
Repositorio local	El repositorio en tu computadora, donde trabajas
Repositorio remoto	Una copia del repositorio en un servidor, aquí GitHub, para respaldarlo y compartirlo
Clonar (`git clone`)	Descargar un repositorio remoto a tu computadora, con todo su historial, y dejarlo conectado al remoto
Push (`git push`)	Subir al remoto los commits que hiciste en tu computadora

`plan-dotnet` va a ser la **evidencia pública** de tu práctica: ahí quedan los ejercicios de las semanas 1 a 5, una carpeta por semana. Por eso va en tu **cuenta personal** de GitHub, no en la organización de DiSí ni en DeployStudioMx.

Paso 1. Comprobar que Git está instalado y configurado

En PowerShell:

```
git --version
git config --global user.name
git config --global user.email
```

- Si `git --version` no responde, instálalo con `winget install Git.Git` y vuelve a abrir PowerShell.
- Si `user.name` o `user.email` salen vacíos, configúralos. Son el nombre y el correo que aparecerán como autor de cada commit; usa el **mismo correo de tu cuenta personal de GitHub**:

```
git config --global user.name "Alexis Ortega"
git config --global user.email "tu-correo-personal@ejemplo.com"
```

Paso 2. Crear el repositorio en GitHub

En el navegador, con la sesión de tu **cuenta personal**:

1. Arriba a la derecha, **+** → **New repository**.
2. Llénalo así:

Campo	Valor	Por qué
Owner	Tu usuario personal	Es tu portafolio
Repository name	`plan-dotnet`	El nombre que usa todo el plan
Description	`Ejercicios de C# y .NET 10: plan de 12 semanas`	Lo primero que lee un reclutador
Visibility	**Public**	Para que se vea como evidencia
Add a README	**Activado**	Así el repositorio no queda vacío y se puede clonar de inmediato
Add .gitignore	**None**	Lo crearás con `dotnet new gitignore` en el laboratorio 1; si GitHub crea uno, ese comando falla porque el archivo ya existe
License	MIT	Indica que otros pueden ver y reutilizar tu código

3. **Create repository**. Copia la dirección que aparece en el botón verde **Code**, pestaña **HTTPS**. Se ve así: `https://github.com/TU-USUARIO/plan-dotnet.git`.

Paso 3. Clonarlo en tu computadora

Elige una carpeta para todos tus repositorios del plan, por ejemplo `C:\Repos`. Una ruta corta y **sin espacios** te ahorra problemas en la terminal.

```
mkdir C:\Repos            # crea la carpeta (solo si no existe)
cd C:\Repos
git clone https://github.com/TU-USUARIO/plan-dotnet.git
cd plan-dotnet
dir                       # debes ver README.md y LICENSE
```

La primera vez que Git se conecte a GitHub, se abrirá el navegador para **autorizar** tu cuenta. Revisa que sea tu **cuenta personal** antes de aceptar.

Si el clone dice `Repository not found`: Git está usando otra cuenta guardada (por ejemplo, la del trabajo). Bórrala con `cmdkey /delete:git:https://github.com` y vuelve a clonar.

Listo: tu ruta del laboratorio 1 es `C:\Repos\plan-dotnet`.

Laboratorio 1. Preparar `plan-dotnet` (15 min)

Requisitos: el repo `plan-dotnet` clonado en tu máquina (laboratorio 0) y el SDK de .NET 10 instalado.

Abre **PowerShell** (no Visual Studio todavía) y entra a la carpeta del repo:

```
cd C:\Repos\plan-dotnet
dotnet --list-sdks                  # debe aparecer una versión 10.x
dotnet new gitignore                # .gitignore estándar de .NET: excluye bin/ y obj/
dotnet new sln -n PlanDotnet        # solución que agrupa los proyectos de todas las semanas
dotnet new console -n Semana01      # crea la carpeta Semana01 con el proyecto
dotnet sln add Semana01/Semana01.csproj
```

Qué acaba de pasar:

- `dotnet new gitignore` crea un `.gitignore` que ya conoce `bin/`, `obj/` y los archivos de Visual Studio.
- `dotnet new sln` crea `PlanDotnet.slnx`. Desde .NET 10, las soluciones nuevas usan el formato **`.slnx`** (XML legible) en lugar del `.sln` clásico que ves en DiSí. Visual Studio 2026 abre los dos.
- `dotnet new console` crea `Semana01/Semana01.csproj` y `Semana01/Program.cs`.
- `dotnet sln add` registra el proyecto en la solución.

Ahora compila y ejecuta:

```
cd Semana01
dotnet build
dotnet run
```

Debes ver `Hello, World!`. Desde la raíz del repo también puedes ejecutar con `dotnet run --project Semana01`.

Hasta ahora no has abierto Visual Studio, y no hace falta. Ese es el punto: entender qué hace el IDE por ti. Ahora sí, abre `PlanDotnet.slnx` en Visual Studio 2026 y confirma que ves el mismo proyecto.

Laboratorio 2. Explorar el proyecto (10 min)

Contesta cada pregunta **por escrito** antes de pasar a la siguiente. La teoría para responderlas está en 10 Teoria de CSharp, sección 2.

1. Abre `Semana01.csproj` con el Bloc de notas. ¿Qué dice `TargetFramework` y qué significa?
2. Abre `Program.cs`. ¿Dónde está el método `Main`? ¿Por qué no lo ves?
3. Entra a `bin/Debug/net10.0/`. Haz una lista de los archivos y escribe junto a cada uno para qué sirve.

4. Ejecuta `dotnet build -c Release`. ¿Qué carpeta nueva apareció?
5. Borra las carpetas `bin` y `obj` y ejecuta `dotnet run`. ¿Qué pasó y por qué?
6. Ejecuta `git status` en la raíz del repo. ¿Aparecen `bin` y `obj`? ¿Por qué no?

Laboratorio 3. Ejercicios (30 min)

Escribe los ejercicios en `Program.cs`, uno debajo del otro, separados por un comentario con su título. El jueves, cuando veas métodos, los vas a reorganizar.

Regla de hoy: predice antes de ejecutar. Antes de correr cada ejercicio, escribe en papel qué crees que va a imprimir. Después compara. Las diferencias son lo que más vas a aprender.

Calentamiento: literales del lunes

Declara tres variables `int` con el mismo número, **200**, escrito de tres formas: en decimal, como literal binario (`0b…`) y como literal hexadecimal (`0x…`). Usa tus respuestas de la clase del lunes. Imprime las tres y comprueba que son iguales.

Pista: si no recuerdas 200 en binario y hex, las respuestas están al final de 11 Clase 0.1 Datos. Intenta primero de memoria.

Ejercicio 1. Ficha de tipos

Imprime una tabla con una fila por cada tipo: `int`, `long`, `double`, `decimal`, `bool`, `char` y `string`. Cada fila debe mostrar:

Columna	Qué debe mostrar
Valor	Un valor de ejemplo tuyo; usa algo de DiSí (un plazo, un monto, una tasa, una inicial)
Tipo .NET	El nombre real del tipo en .NET (por ejemplo, `System.Int32`)
Bytes	Cuántos bytes ocupa
Mínimo y máximo	Solo para los tipos numéricos

Pistas, no soluciones:

- Todo valor tiene el método `GetType()`.
- El operador `sizeof(tipo)` devuelve el tamaño en bytes de los tipos numéricos, `bool` y `char`.
- Los tipos numéricos tienen `MinValue` y `MaxValue`.
- Para alinear columnas, la interpolación acepta un ancho: `{valor,-12}` alinea a la izquierda en 12 caracteres.

Pregunta para pensar: intenta usar `sizeof(string)`. ¿Qué te dice el compilador? ¿Por qué tiene sentido, sabiendo que `string` es un tipo por referencia? (Lo verás a fondo la semana 2.)

Ejercicio 2. Provocar un desbordamiento

1. Guarda `int.MaxValue` en una variable e imprímela.
2. **Predice** qué imprime al sumarle 1. Escríbelo en papel.
3. Súmale 1 e imprime el resultado. ¿Coincidió con tu predicción? Explica el resultado con lo que viste el lunes (complemento a dos).
4. Repite la suma dentro de `checked`. ¿Qué pasa ahora? Anota el nombre exacto de la excepción.
5. Intenta escribir directamente `int.MaxValue + 1`, sin variable, dentro de `checked`. ¿Compila? Anota el código de error que te da el compilador y explica por qué es distinto del paso 4.

Pista: la diferencia entre el paso 4 y el 5 está en "Desbordamiento" de 10 Teoria de CSharp, sección 3.

Pregunta para pensar: en un sistema de crédito, ¿qué es peor: un programa que truena con `OverflowException` o uno que sigue corriendo con un monto negativo? ¿Por qué?

Si te sobra tiempo

Haz lo mismo del ejercicio 2 con `byte`: guarda 255 y súmale 1. ¿Qué número esperas? Pista: el resultado de sumar dos `byte` en C# no es `byte`; averigua de qué tipo es y por qué necesitas un cast.

Cierre (10 min)

1. **Commit y push**, desde la raíz de `plan-dotnet`:

   ```
   git add .
   git status        # revisa que NO aparezcan bin/ ni obj/
   git commit -m "Semana 1: primer proyecto, ficha de tipos y desbordamiento"
   git push
   ```

2. **Feynman (2 min, en voz alta):** explica qué pasa desde que escribes `dotnet run` hasta que la CPU ejecuta tu código. Usa las palabras Roslyn, IL, CLR y JIT.
3. **Glosario:** pasa a 08 Glosario los términos de hoy: SDK, runtime, CLR, IL, JIT, ensamblado, top-level statements, `checked`, overflow.
4. **Bitácora:** anota en 01 Bitacora dónde te trabaste y qué te sorprendió.

Está terminado cuando: compila sin advertencias, cada línea la puedes explicar en voz alta y está en GitHub.

Si algo falla

Síntoma	Causa probable	Qué hacer
`dotnet` no se reconoce	PowerShell se abrió antes de instalar el SDK	Cierra y vuelve a abrir PowerShell
`dotnet --list-sdks` no muestra 10.x	Falta el SDK de .NET 10	Instálalo desde el Visual Studio Installer o con `winget install Microsoft.DotNet.SDK.10`
`Couldn't find a project to run`	Estás en una carpeta sin `.csproj`	Entra a `Semana01` o usa `--project Semana01`
`git status` muestra `bin/` y `obj/`	Falta el `.gitignore` o está en otra carpeta	Corre `dotnet new gitignore` en la raíz del repo
`git clone` dice `Repository not found`	Git usa otra cuenta guardada	`cmdkey /delete:git:https://github.com` y vuelve a clonar
`dotnet new gitignore` dice que el archivo ya existe	GitHub creó un `.gitignore` al crear el repo	Bórralo (`del .gitignore`) y vuelve a correr el comando
`dotnet sln add` dice que no existe el archivo de solución	Falta crear la solución antes de agregarle proyectos	Corre `dotnet new sln -n PlanDotnet` y repite el `sln add`
Windows (**Smart App Control**) bloquea el programa: "no podemos confirmar quién publicó Semana01.dll" o `FileLoadException`	Smart App Control bloquea todo `.exe` y `.dll` sin firma digital, incluido lo que compilas tú. No se arregla desde el proyecto	En una PC **personal**: Seguridad de Windows → Control de aplicaciones y navegador → Smart App Control → **Desactivado** (en muchas versiones ya no se puede reactivar sin reinstalar Windows). En una PC **de la empresa**: no lo cambies; usa tu PC personal o GitHub Codespaces
Visual Studio no abre el `.slnx`	Versión de Visual Studio antigua	Actualiza Visual Studio 2026 desde su Installer

Preguntas de entrevista

1. ¿Qué diferencia hay entre `dotnet build` y `dotnet run`?
2. ¿Qué hay en `bin` y qué hay en `obj`? ¿Por qué no se suben a Git?
3. ¿Qué pasa cuando un `int` se desborda, con y sin `checked`?
4. ¿Por qué `sizeof(string)` no compila?
5. ¿Qué genera el compilador cuando usas top-level statements?

Tarjetas de repaso

Formato del plugin Spaced Repetition de Obsidian.

Pregunta	Respuesta
¿Qué comando crea un proyecto de consola?	dotnet new console -n Nombre
¿Qué comando crea el .gitignore de .NET?	dotnet new gitignore
¿Qué formato de solución crea .NET 10 por defecto?	.slnx (XML), en lugar del .sln clásico
¿Diferencia entre dotnet build y dotnet run?	build compila a IL en bin/; run compila si hace falta y ejecuta
¿Qué hay en obj/?	Archivos intermedios de la compilación; no se tocan ni se suben a Git
¿Qué devuelve 5.GetType()?	System.Int32
¿Cuánto vale sizeof(decimal)?	16 bytes
¿Qué pasa al sumar 1 a int.MaxValue sin checked?	Se desborda en silencio y da int.MinValue (−2,147,483,648)
¿Qué excepción lanza un desbordamiento dentro de checked?	System.OverflowException
¿Por qué checked(int.MaxValue + 1) no compila?	Son constantes: el compilador detecta el desbordamiento al compilar (error CS0220)
¿Por qué sizeof(string) no compila?	string es un tipo por referencia y su tamaño no es fijo
¿Qué es un repositorio de Git?	Una carpeta cuyo historial registra Git como commits, para ver quién cambió qué y regresar a versiones anteriores
¿Repositorio local vs. remoto?	El local está en tu computadora; el remoto en un servidor como GitHub, para respaldar y compartir
¿Qué hace git clone?	Descarga un repositorio remoto con todo su historial y lo deja conectado al remoto
¿Qué hace git push?	Sube al remoto los commits hechos en tu computadora